

AI 工牌 1.0 端云通信接口规范 · v4

- 状态：已定稿；协议参数已冻结（ODM 已确认）。
- v5 修订：新增「分组多语言广播」（§ 3.3 ⑪ broadcast_ack 回执、§ 4.2 ⑧ broadcast 下行、§ 5.5 广播时序）。
- v4 修订：写死键→状态迁移表（补 staff_qa 内问答键行为、退出问答返回进入前状态）、scope_change 竞态收口、fixed 模式显示与 reset 回执、query_scope 派生、staff_qa 状态机约束、命名统一。
- 权威来源：《AI工牌1.0产品需求文档（0804）》§ 6.4/ § 9.3/ § 10、《软件端顶层架构总纲》§ 4/ § 6/ § 10。
- 适用对象：ODM 端侧固件（ESP32-P4 + C5 + IA8201）与甲方云端后端联调。
- 传输：全双工 WebSocket（WSS）。音频走二进制帧，控制消息走 JSON 文本帧。

1. 连接与鉴权

1.1 连接方式

- 协议：WebSocket over TLS（wss://）。
- 建连时机：短按翻译键触发 → 建连（scope=translate）；无活跃会话时短按问答键触发 → 建连（scope=staff_qa）。会话结束（再次短按翻译键 / 空闲超时）→ 断开。会话级短连，不维持长连接。
- 端点：wss://<host>/ws?device_id=<id>&token=<t>&fw_version=<v>&proto_ver=1
- proto_ver：协议版本（当前 1），帧结构演进协商用。

1.2 握手鉴权

参数	类型	说明
device_id	string	设备唯一标识，云端签发
token	string	设备短期凭证（demo 静态；真实上线短期轮换）
fw_version	string	端侧主固件版本（P4）
proto_ver	int	协议版本

- 握手云端用 device_id + token 鉴权，失败拒绝握手（HTTP 403）。
- 鉴权成功云端由设备身份推导 customer_id / store_id；会话内消息不再带 token、不接受租户字段。
- ⚠ token 过期：真实上线建议「断开重连重新握手」，demo 静态 token 不处理。

1.3 session_id 与 turn_id

- session_id：一次翻译会话的唯一标识，设备生成（UUID），会话内所有消息带同一值。断线重连必须生成新 session_id。
- turn_id：会话内一轮对话（一句话）的唯一标识，设备生成（UUID），随新 utterance 更换。
- 规则：同一 turn 内，音频分片、对应 TTTS、对应 stop_tts 使用同一 turn_id；打断后新 turn 换新 turn_id，stop_tts 指向被停的旧 turn_id。sequence 仅在 turn 内从 0 递增，不跨 turn 复用。

1.4 会话业务域 scope 与语言对模式 translation_mode (v3 核心)

两个独立概念，勿混：

概念	取值	切换消息	是否清 visitor_language
scope （会话业务域）	translate / staff_qa	scope_change （会话内）	不清（PRD § 9.3 员工问答不清空）
translation_mode （语言对模式）	auto / fixed	translation_mode_change	清（PRD § 6.4.10 模式切换重置）

query_scope 由 scope 派生：translate → customer、staff_qa → staff。端侧只传 scope，云端从 scope 派生 query_scope（不信任端侧传的 query_scope，冲突以 scope 为准）。

键 → 状态迁移表（写死，ODM 照此实现）：

当前状态	按键	动作
待机（无会话）	短按翻译键	建连，scope=translate
待机（无会话）	短按问答键	建连，scope=staff_qa
translate 会话中	短按问答键	scope_change → staff_qa（保留 visitor_language）
staff_qa 会话中	短按翻译键	scope_change → 返回进入前状态（从 translate 进→回 translate；从待机直进→结束会话回待机），保留 visitor_language
staff_qa 等待中	短按问答键	取消本轮提问，保持 staff_qa（可继续提问）
staff_qa 播放中	短按问答键	stop_tts 停止当前答案播放，保持 staff_qa
translate 会话中	短按翻译键	结束会话（断开，清 visitor_language）
任一状态	空闲超时	结束会话（断开，清 visitor_language）

staff_qa 状态机约束：

- staff_qa 内不执行 LID、不建立/更新 visitor_language（冻结语言记忆）。
 - staff_qa 内 translation_mode_change 非法，云端忽略（SW-SKB-009 员工问答不得修改 translation_mode）。
-

2. 帧结构总览

帧类型	方向	编码	frame_type
音频分片	端 → 云	二进制帧	0x01
TTS 音频	云 → 端	二进制帧	0x81
控制消息	双向	JSON 文本帧 (UTF-8)	—

- 字节序：帧头多字节整数（sequence/timestamp 等）大端 big-endian；音频 PCM 采样值 小端 little-endian（与 WAV 一致）。
- frame_type 高 4 位分方向：0x0x 上行、0x8x 下行。

2.1 上行音频帧（0x01）

偏移	长度	字段
0	1 byte	frame_type = 0x01
1	1 byte	turn_id_len = N (1~255)
2	N bytes	turn_id (UTF-8, UUID)
2+N	4 bytes	sequence (uint32 大端, turn 内从 0 递增)
6+N	8 bytes	timestamp (uint64 大端, 会话内相对毫秒, 自建连起)
14+N	1 byte	source_side: 0=wearer 1=counterparty 2=uncertain
15+N	1 byte	input_source: 0=badge_near 1=badge_far 2=headset_mic 3=uncertain
16+N	剩余	audio_payload (裸 PCM: PCM16 · 16kHz · 单声道 · little-endian, 不含 WAV/RIFF 头)

input_source 以帧级为准（每帧携带）；eou/session_init 的 input_source 为会话/句级参考，与帧级冲突时以帧级为准。

2.2 下行 TTS 帧（0x81）

偏移	长度	字段
0	1 byte	frame_type = 0x81
1	1 byte	turn_id_len = N
2	N bytes	turn_id (UTF-8, UUID)
2+N	剩余	tts_audio (RIFF/WAV: 44 字节 WAV 头 + PCM16 · 16kHz · 单声道 · little-endian, 流式分片下发)

流结束由 JSON tts_end 标记，TTS 帧本身不携带结束标记。0x81 帧只带 turn_id，不带 session_id（单会话连接内由 turn_id 唯一定位）。

2.3 JSON 控制消息

- 文本帧 UTF-8 JSON，以 type 区分；字段 snake_case；未出现字段视为 null/缺省。
- JSON 控制消息统一带 session_id（0x81 二进制帧例外）。

3. 上行消息

3.1 会话初始化 `session_init`（建连后首条，必发）

```
{
  "type": "session_init",
  "session_id": "s1",
  "scope": "translate",
  "staff_language": "ja-JP",
  "translation_mode": "auto",
  "lang_pair": "zh-ja",
  "headset_state": "disconnected",
  "input_source": "badge_far",
  "wifi_band": "5GHz",
  "network_state": "good",
  "config_version": 123,
  "candidates": ["ja-JP", "zh-CN", "en-US", "ko-KR"]
}
```

字段	说明
<code>scope</code>	<code>translate</code> / <code>staff_qa</code> （会话初始业务域）
<code>staff_language</code>	当前员工语言（双向翻译锚点）
<code>translation_mode</code>	<code>auto</code> / <code>fixed</code> （翻译语言对模式）
<code>lang_pair</code>	<code>fixed</code> 时填写（如 <code>zh-ja</code> ）； <code>auto</code> 可空
<code>headset_state</code>	<code>disconnected</code> / <code>connected</code> / <code>input_error</code> / <code>output_error</code>
<code>input_source</code>	当前主输入通道
<code>wifi_band</code> / <code>network_state</code>	网络状态（运营分析用）
<code>config_version</code>	设备当前配置版本号（pull 比对用）
<code>candidates</code>	当前门店候选语种集

`query_scope` 由 `scope` 派生（`translate`→`customer` / `staff_qa`→`staff`），端侧不传，云端从 `scope` 推导。

3.2 二进制音频帧（`0x01`）

见 § 2.1。持续流式上传，每帧一个分片。

3.3 JSON 控制消息

① 尾包 `eou` —— 该 `turn` 的最后一条消息，之后才允许开始新 `turn`：

```
{"type": "eou", "session_id": "s1", "turn_id": "t1", "source_side": "counterparty",
  "last_sequence": 23, "utterance_duration_ms": 1500, "input_source": "badge_far"}
```

字段	说明
<code>source_side</code>	该句说话方
<code>last_sequence</code>	该 turn 最后分片 sequence 号（收齐校验）
<code>utterance_duration_ms</code>	有效语音时长（质量门控 <code>min_utt_duration_ms</code> ）
<code>input_source</code>	该句实际输入通道（与帧级冲突时帧级为准）

② 打断上报 `stop_tts` —— 端侧本地打断后异步上报，`turn_id` 指向被停的旧 turn：

```
{"type":"stop_tts","session_id":"s1","turn_id":"old_t1","reason":"voice"}
```

`reason` 枚举：voice（语音抢占）/ button（按键停止）。

③ 睡眠上报 `sleep_notice`：

```
{"type":"sleep_notice","session_id":"s1"}
```

④ 按键事件 `button_event`：

```
{"type":"button_event","session_id":"s1","button":"ok"}
```

`button:"ok"` 为 VKB（游客知识库）候选确认：游客提问翻译成日语后，云端先撞 VKB，命中则「嘀嘀」提示服务人员，服务人员按 OK 确认后云端把答案反向翻译成游客语言播放；未命中/超时则回退播翻译原文（见 § 5.1）。

⑤ 播放事件 `playback_event`：

```
{"type":"playback_event","session_id":"s1","turn_id":"t1","event":"completed"}
```

`event`：started / completed（打断以 `stop_tts` 为准，不要求发 interrupted）。

⑥ 业务域切换 `scope_change` —— 员工进入/退出问答，会话内、不清 `visitor_language`：

```
{"type":"scope_change","session_id":"s1","scope":"staff_qa"}
```

`query_scope` 由 `scope` 派生，端侧不传。

⚠ in-flight 竞态：翻译 TTS 播放中或处理中切 `scope`，端侧先本地停播 + 发 `stop_tts`(旧 `turn_id`) 再发 `scope_change`；云端收到 `scope_change` 后作废当前翻译 turn（不再下发该 turn 的 TTS/tts_end）。

⑦ 语言对模式切换 `translation_mode_change` —— AUTO↔固定语言对，清 `visitor_language`：

```
{"type":"translation_mode_change","session_id":"s1","translation_mode":"fixed","lang_pair":"zh-ja"}
```

⑧ 耳机事件 **headset_event** —— mid-session 耳机插拔/异常上报:

```
{"type":"headset_event","session_id":"s1","headset_state":"output_error"}
```

⑨ 配置拉取 **config_pull** :

```
{"type":"config_pull","session_id":"s1","config_version":123}
```

⑩ 心跳 —— WS 原生 ping/pong (非应用层), 间隔见 § 9。

⑪ 广播回执 **broadcast_ack** —— 收到广播并确认后上报:

```
{"type":"broadcast_ack","session_id":"s1","broadcast_id":42}
```

字段	说明
<code>broadcast_id</code>	广播 ID (广播下行消息里携带, 回执据此定位)

广播播放 / 显示完成后, 用户按 OK 键确认, 设备上报 `broadcast_ack`, 云端记录回执 (后台回执看板)。

4. 下行消息

4.1 二进制 TTS 帧 (0x81)

见 § 2.2。流式下发, 首帧即播。

4.2 JSON 控制消息

① TTS 流结束 **tts_end** :

```
{"type":"tts_end","session_id":"s1","turn_id":"t1"}
```

⚠ 打断竞态: 端侧发出 `stop_tts` 后, 云端停发该 `turn` 后续分片并不再补发 `tts_end`; 端侧发出 `stop_tts` 后忽略迟到的 `tts_end` (按 `turn_id` 匹配丢弃)。

② 错误 **error** :

```
{"type":"error","session_id":"s1","turn_id":"t1","code":"E_LID_LOW","message":"请再说一遍","fallback_action":"ask_repeat"}
```

turn_id 会话级错误 (E_AUTH/E_NET/E_SESSION) 可缺省; session_id 必带。

- ③ 文本 **final_text** / **partial_text** —— ⚠️ 端侧不显示, 仅调试/日志:

```
{"type": "final_text", "session_id": "s1", "turn_id": "t1", "text": "你好"}
```

- ④ 知识库答案 **answer** —— 员工知识库 (skb) 答案, TTS 音频随后用 0x81 帧下发:

```
{"type": "answer", "session_id": "s1", "turn_id": "t1", "text": "早餐时间为 7:00-10:00", "source_refs": ["早餐FAQ@v1#42"]}
```

- ⑤ 语言状态 **language_state** —— 合并了 lid_confidence, 设备屏幕显示用:

```
{"type": "language_state", "session_id": "s1", "turn_id": "t1", "visitor_language": "zh-CN", "lid_language": "zh-CN", "confidence": 0.92, "state": "locked"}
```

state: unknown / locked / updating / reset / conflict。

⚠️ translation_mode=fixed 时: visitor_language 显示为 lang_pair 对端语种 (固定, 不跑 LID); translation_mode_change 切换后云端回 language_state(state=reset), 端侧据此刷新屏幕。

- ⑥ 配置下发 **config_update**:

```
{"type": "config_update", "session_id": "s1", "config_version": 124, "config": {...}}
```

config 结构见 § 7。

- ⑦ 输出路由 **output_route** —— 耳机 P1 后置, 字段预留:

```
{"type": "output_route", "session_id": "s1", "turn_id": "t1", "route": "speaker"}
```

耳机路由规则: output_route 必须先于 TTS 首帧到达。

- ⑧ 广播 **broadcast** —— 管理员向群成员下发的一条广播 (设备级消息):

```
{"type": "broadcast", "broadcast_id": 42, "text": "Thông báo an toàn", "target_language": "vi-VN"}
```

字段	说明
broadcast_id	广播 ID (回执 broadcast_ack 用)
text	该设备母语的译文 (墨水屏显示)
target_language	译文语言

- 广播是设备级消息（管理员 → 全体群成员），不绑定翻译会话，`session_id` 可缺省。

- 随后译文 TTS 用 `0x81` 帧下发，`turn_id = "bc"+broadcast_id`（如 `bc42`），以 `tts_end` 结束。

- 收到后设备显示译文 + 播放 TTS；用户按 OK 键回执 `broadcast_ack`。

5. 时序

5.1 翻译链路（双向）

设备 —(握手鉴权)—> 云端
设备 —`session_init`—> 云端 (`scope/query_scope/staff_language/translation_mode/候选语种`)
设备 —`0x01` 音频分片×N—> 云端 (流式, `source_side + input_source`)
(云端对已到达分片流式预识别, `eou` 仅触发尾包收口与最终翻译)
设备 —`eou`—> 云端 (`last_sequence` 收齐校验)
云端: 说话方判定 → LID → ASR → MT → TTS
云端 —`0x81` TTS 分片×N—> 设备 (首帧即回)
云端 —`tts_end`—> 设备 (流结束)
设备 —`playback_event(completed)`—> 云端

- 员工发言 (`source_side=wearer`) → 翻译为 `visitor_language` 外放；游客发言 (`counterparty`) → LID 判向、翻译为 `staff_language`。
- 固定语言对 (`translation_mode=fixed`) → 跳过 LID，按 `lang_pair` 双向翻译。

VKB 自动应答 (`scope=translate`, `auto` 模式游客发言)：

游客发言翻译成日语后，云端先用日语译文撞 VKB 知识库，再决定播什么：

游客 —`0x01` 音频×N—> 云端 (`auto` 一步翻译)
云端: 翻译成日语译文 → 撞 VKB 知识库
├ 命中:
│ 云端 —`0x81`「滴滴」提示音—> 设备 (不播翻译原文)
│ 云端 —`language_state`—> 设备
│ 服务人员按 OK → `button_event(button:ok)` —> 云端
│ 云端: 答案反向翻译成游客语言 → `0x81` 播放 + `tts_end`
│ 或 5s 超时未确认 → 回退播翻译原文(`0x81`) + `tts_end`
└ 未命中:
│ 云端 —`0x81` 翻译原文 + `tts_end`—> 设备

- 提示音与答案均走 `0x81` TTS 通道（耳机输出路由为独立后置项，本阶段不区分耳机/扬声器）。
- 命中后翻译原文暂不播，仅发「滴滴」提示 + `language_state`；超时/未命中才回退播原文。

5.2 员工知识库问答链路 (`scope=staff_qa`)

设备 —`scope_change(staff_qa)`—> 云端 (同一 `session`, 保留 `visitor_language`)
设备 —`0x01` 音频分片×N—> 云端
设备 —`eou`—> 云端
云端: ASR(`staff_language`) → 向量检索 → 受控 RAG → LLM 受限生成 → 校验
云端 —`answer(JSON: text + source_refs)`—> 设备

云端 —0x81 TTS 分片×N→ 设备（答案 TTS）
云端 —tts_end→ 设备
设备 —scope_change(translate) / 短按翻译键退出问答→ 云端

- staff_qa 内不执行 LID、不更新 visitor_language；translation_mode_change 忽略。

5.3 TTS 打断（端侧本地，≤100ms，不等云端）

设备播放中：检测到新语音 / 按键
├ 本地：停播 + 清缓存 + 复位拾音（≤100ms，断网可用）
└ 异步：stop_tts(旧 turn_id) → 云端（停合成 + 释放 + 记账，不再补发 tts_end）

- 严禁以断开 WSS 实现打断。
- 迟到 TTS 分片/tts_end 按 turn_id 丢弃；云端容忍中途停流。

5.4 断线重连

- 网络断线 = 会话结束、visitor_language 清空；重连必须生成新 session_id 重新握手，回到 UNKNOWN，设备自动恢复不需重启。
- ⚠️ 1.0 不加「会话恢复窗口」，2.0 再评估。

5.5 广播链路（管理员 → 群成员，一对多）

广播由管理员在后台触发，云端按成员母语去重翻译 + TTS，对在线成员实时推送、离线成员重连后补拉：

管理员 —REST 发广播（群 + 源文本 + 源语言）→ 云端
云端：查群成员 → 按 target_language 去重 → 每种语言翻译一次 + TTS 一次
├ 在线成员：broadcast(JSON) → 0x81 TTS 分片 → tts_end（实时推，标已投递）
└ 离线成员：标「未投递」，重连后补拉
设备：显示译文 + 播放 TTS → 用户按 OK
设备 —broadcast_ack(broadcast_id)→ 云端（回执）

离线补拉（发广播时设备不在线）：

设备 —session_init→ 云端
云端：查该设备未投递广播 → broadcast → 0x81 TTS → tts_end

- 同语言成员共享同一份译文 + TTS（去重，省翻译 / 合成成本）。
- 广播不依赖翻译会话；设备只需在线（有 WSS 连接）即可收。
- 补拉时云端按已存译文重新合成 TTS 下发（不落音频存储）。

连接生命周期与实时性（重要）：广播不保证离线实时到达。

- 「在线」= 设备当前有 WSS 连接（翻译会话中，或按键唤醒后、会话结束前）；「离线」= 设备无 WSS 连接（待机 / Deep Sleep）。

- 设备待机时进入 Deep Sleep，不维持长连接、不维持心跳（保证 7 - 10 天待机），故离线设备在广播发生时收不到，标记「未投递」。

- 离线设备下次唤醒 / 建连（`session_init`）后，云端补拉未投递的广播。
- 功耗是硬指标，广播是「通知」非「即时消息」，延迟到「员工下次唤醒」可接受；需秒级实时的紧急场景后续单独设计唤醒机制，不纳入本广播。

6. 错误码表

错误码	场景	设备端行为
E_AUTH	鉴权失败	提示 + 重新鉴权（建议断开重连）
E_NET	网络断开/超时	重连；提示音；本地打断仍可用
E_LID_LOW	LID 低置信且非白名单	播「请再说一遍」；不外放译文
E_PROVIDER	供应商全挂（已降级仍失败）	播「服务暂不可用，稍后再试」
E_ASR_FAIL	ASR 失败	提示重试 / 云端切备选
E_QA_NO_MATCH	知识库无匹配	播「暂无明确说明」兜底
E_QA_OUT_OF_SCOPE	范围外问题	播兜底话术（可配置）
E_CONFLICT	输入来源/语言冲突、重叠讲话	提示「请依次说话/重新说」；不播放
E_SAME_LANG	双方同语言、游客说 <code>staff_language</code>	不翻译 / 直通 / 提示重说
E_SESSION	会话被云端强制终止	结束当前会话，回待机

7. 配置下发（`config_update`）

类型	字段	生效时机
静态配置	<code>staff_language</code> / <code>lang_pairs</code> / <code>lid_theta</code> / <code>lid_theta_super</code> / <code>min_utt_duration_ms</code> / <code>kws_whitelist</code> / <code>brand_words</code> / 数字过滤 / <code>idle_timeout_ms</code> / 音量 / <code>endpoint</code>	设备唤醒 pull 时生效
会话动态	兜底话术更新 / 强制扬声器	会话中 <code>config_update</code> 推，低频

`config` 示例：

```
{
  "staff_language": "ja-JP",
  "lang_pairs": ["zh-ja", "en-ja", "ko-ja"],
  "lid_theta": 0.8,
  "lid_theta_super": 0.95,
  "min_utt_duration_ms": 500,
  "kws_whitelist": ["OK", "はい", "ありがとう"],
  "brand_words": ["iPhone", "LV"],
```

```
"numeric_filter": true,
"idle_timeout_ms": 20000,
"volume": {"speaker": 60, "headset": 40},
"endpoint": "wss://..."
}
```

⚠️ LID 策略仍缺「第三语言处理」「软锁定连续 N 次/更新阈值」等可配置项，联调前在后台 LID 策略域补齐（架构 §11 语言策略域）。

设备唤醒/重连发 `config_pull` 拉最新（带 `config_version`，只拉变更）；会话中低频变更用 `config_update` 推。

8. OTA 升级（独立通道）

- 通道：独立 HTTPS 文件下载，不经 WSS 音频通道。
- 下发：云端通过 `config_update` 下发 ota 信息（目标模块、目标版本、包 URL、SHA256、依赖/顺序）。
- 上报：设备用 `button_event` 或专用 `ota_event`（预留）上报进度/结果。
- 目标模块：ESP32-P4 主固件 / ESP32-C5 无线固件 / IA8201 参数，三类独立管理。
- ⚠️ 具体 ota 字段、升级顺序、失败回滚、断电恢复，由 ODM 定义，联调前冻结。

9. 已冻结参数清单（ODM 已确认）

参数	冻结值	说明
采样率	16kHz	端侧/云端一致
音频编码	PCM 16bit 单声道；上行裸 PCM（不含头），下行 TTS 带 44 字节 WAV 头（RIFF，小端 little-endian）	v1 固定；未来 Opus 需协商
上行帧长	100ms	每 0x01 帧约 100ms 音频
TTS 下行分片	≤8KB（约 100ms）	首帧即播
单帧上限	≤8KB	音频分片上限
心跳	10s	WS ping/pong，连续 2-3 次无 pong 判死 → 断线重连
会话空闲超时	20s	无有效语音/按键自动结束并断开
LID 阈值 θ / θ_{super}	不适用（移除）	auto 模式用一步 auto-detect，不依赖置信度
背压/流控	1.0 暂不做	仅约定断连清理会话缓冲

10. 甲方硬性要求（TTS 打断三条）

1. 端侧打断四步 ≤100ms、断网可用（产品级验收）。
2. P4↔IA8201 必须有 AEC 播放参考通路（语音抢占打断的前提）。

3. 迟到的 TTS 分片按 turn_id 丢弃、云端容忍中途断流不记错。

附录：与架构文档的对应关系

本规范章节	架构文档
§ 1 连接鉴权 / scope	§ 4、§ 8.2、§ 9.5
§ 2 帧结构	§ 4、附录 B
§ 3/ § 4 消息	§ 4 端云接口契约
§ 5 时序	§ 3.1、§ 6.5、§ 10.3
§ 6 错误码	§ 10.2
§ 7 配置	§ 11
§ 8 OTA	§ 11.5
§ 9 已冻结参数	0804 § 10.1
§ 10 硬性要求	§ 6.7